



KubeCon



CloudNativeCon

North America 2025

Simplifying Cloud-Native App Testing across Environments

Artur CIOCANU

Laurent BROUDOUX

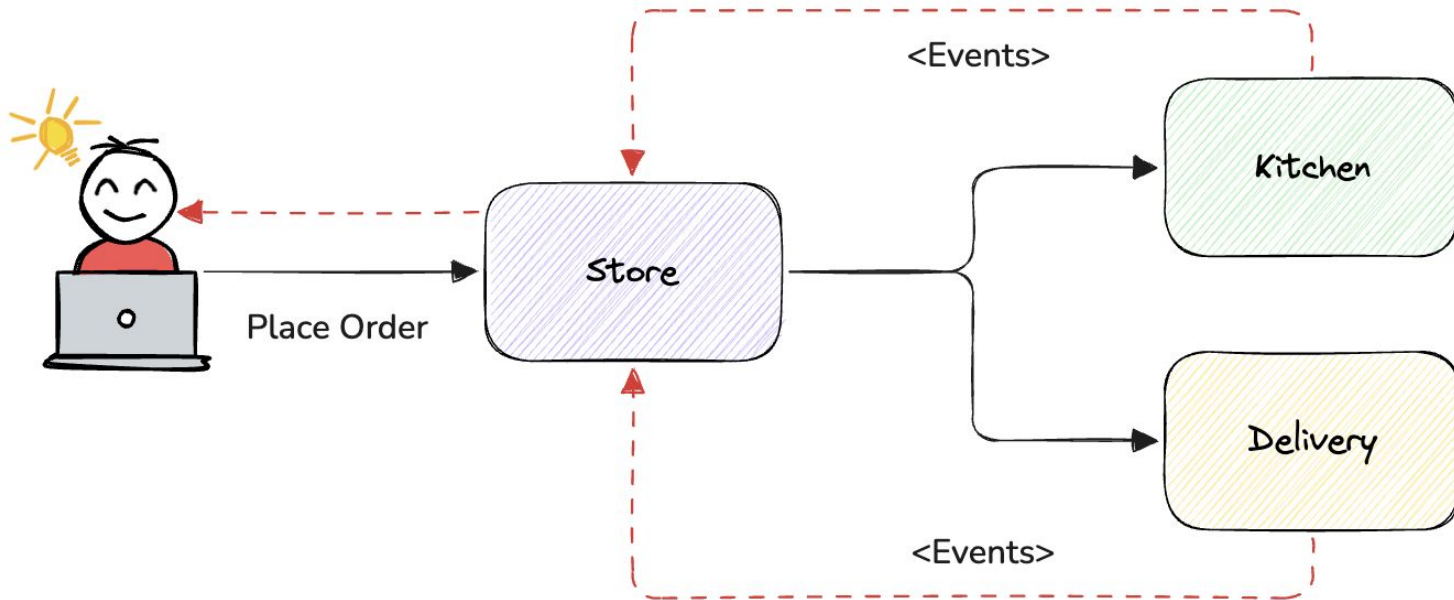
#dapr #microcks



Demo #1



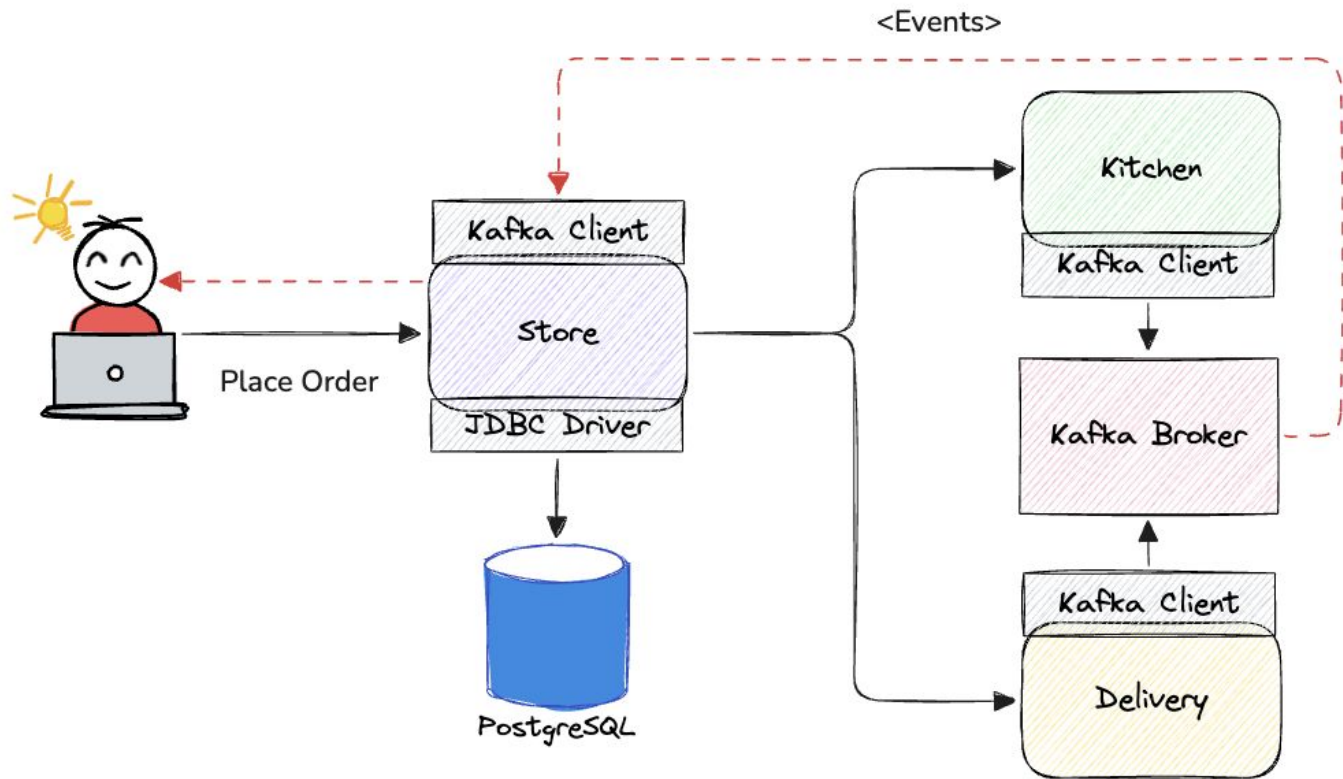
Cloud Native Pizza Store



<https://github.com/salaboy/pizza>

<https://www.cncf.io/blog/2025/06/25/cloud-native-app-local-development-made-easy-with-microcks-and-dapr/>

Cloud Native Pizza Store (with infra)



Who we are!



Artur Ciocanu

@artur-ciocanu

Senior Computer Scientist @ Adobe Inc
Dapr Java SDK Maintainer

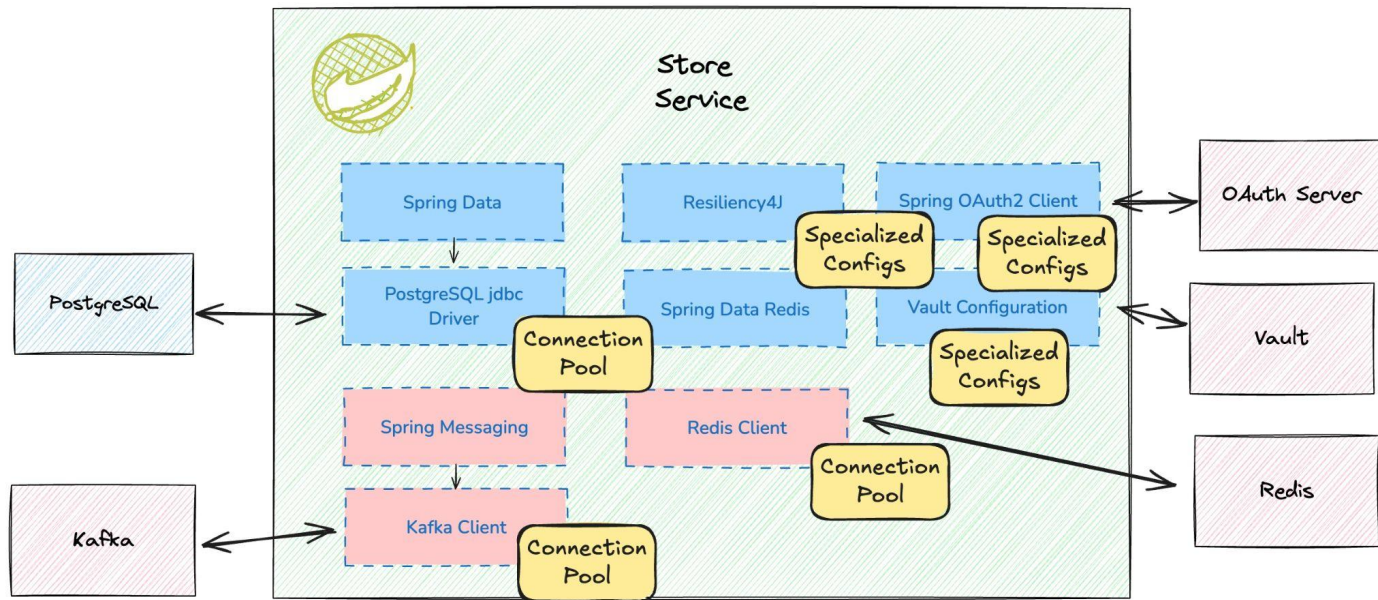


Laurent Broudoux

@lbroudoux

Cloud native Architect @ Postman
Microcks founder

What we need for an Enterprise Application

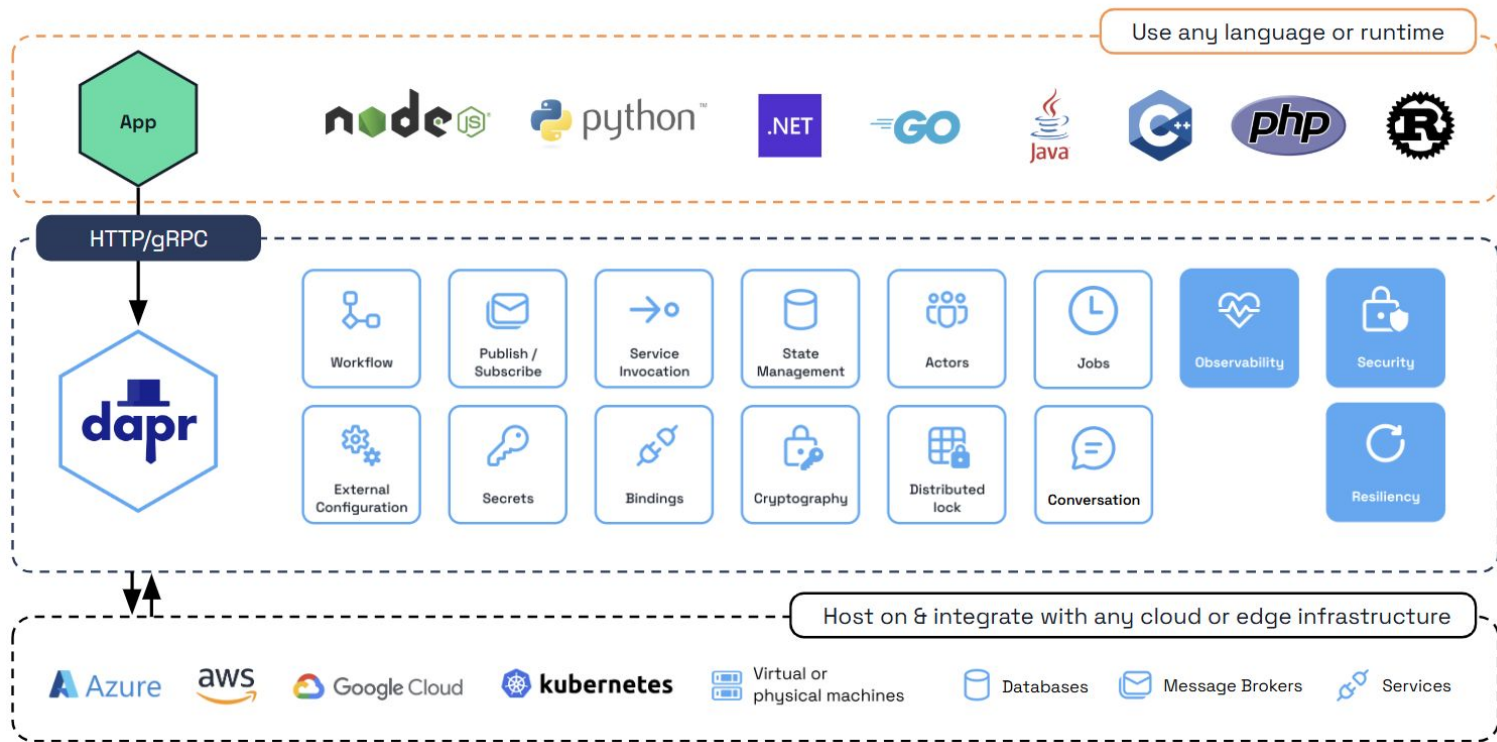




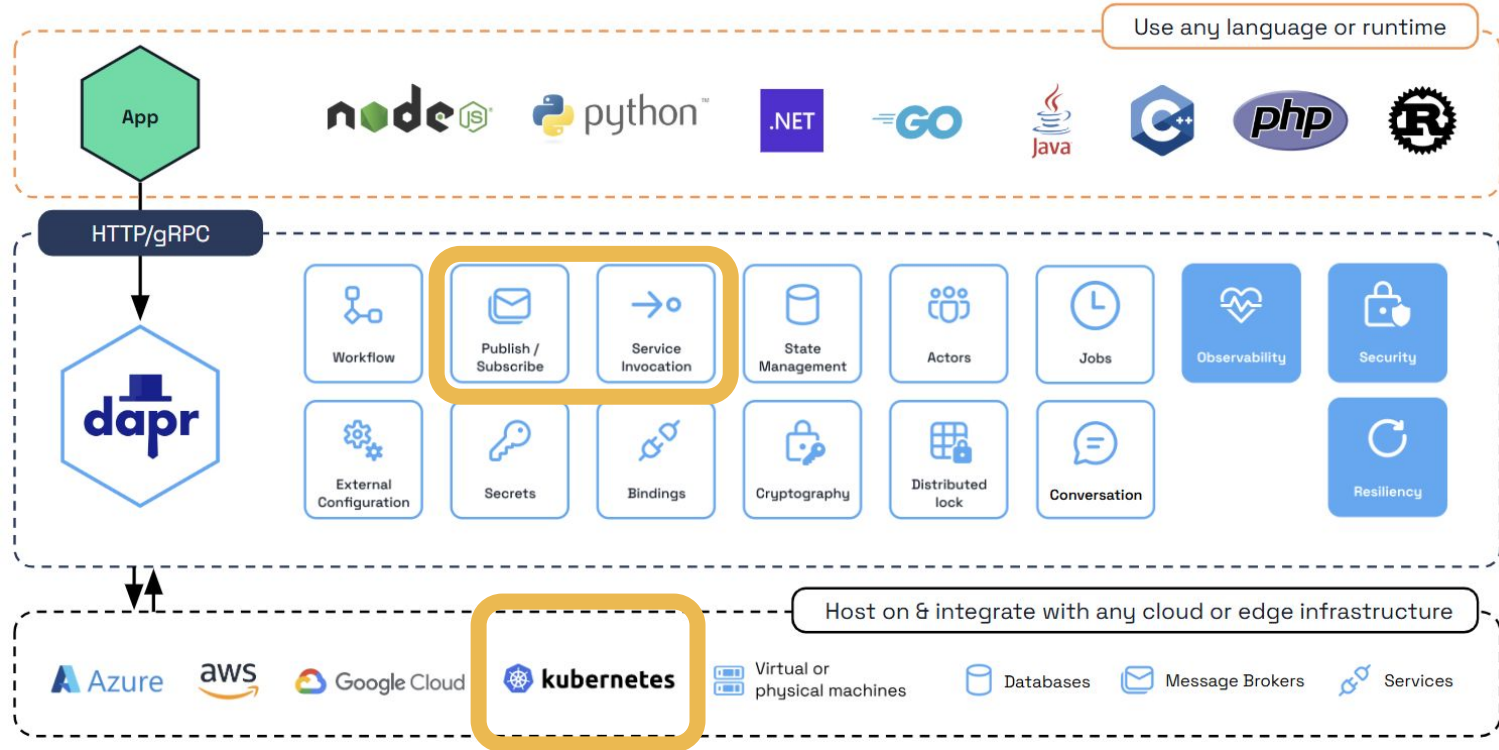
- **Distributed **A**pplication **R**untime**
- CNCF Graduated Project
- Multi language SDKs
- Case Study

<https://www.cncf.io/case-studies/datagalaxy/>

Dapr (<https://dapr.io>)



Let's start with the basics



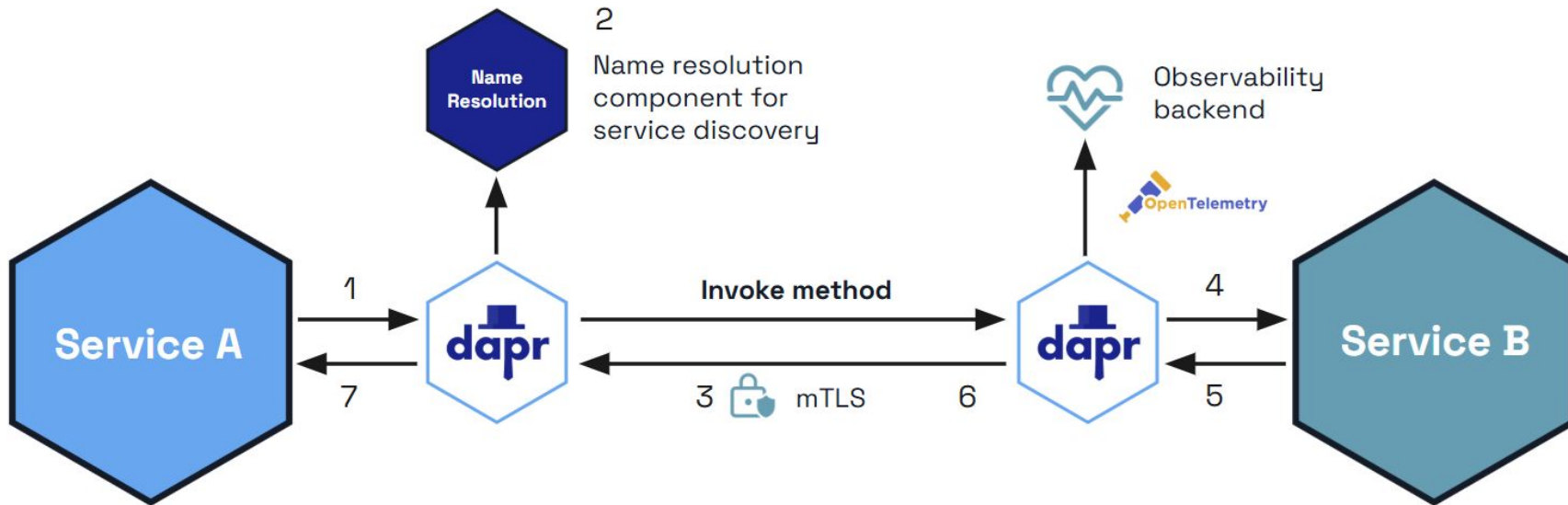
KubeCon



CloudNativeCon

North America 2025

Service-to-Service Invocation (Store to Kitchen and Delivery)



POST

<http://localhost:3500/v1.0/invoke/checkout/method/order>

POST

<http://localhost:5100/order>



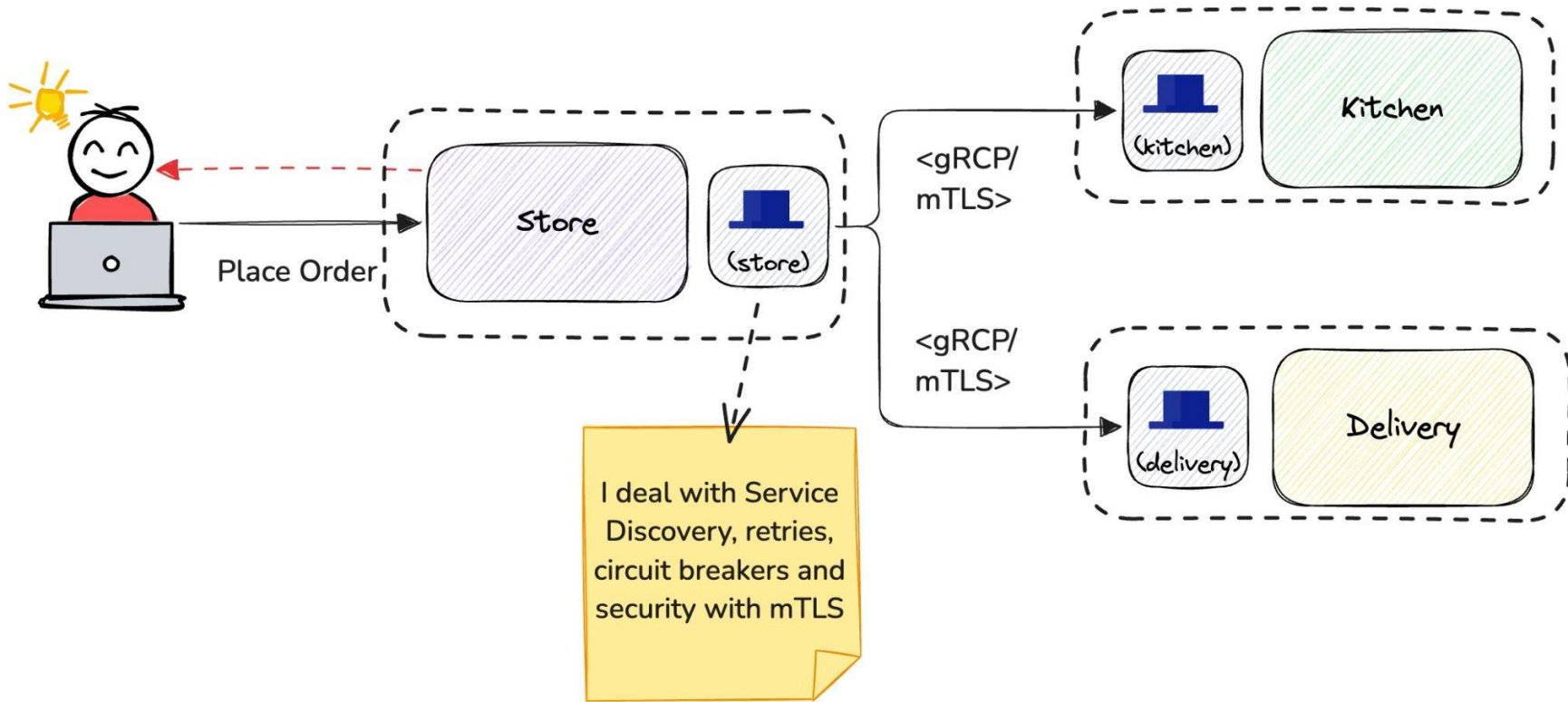
KubeCon



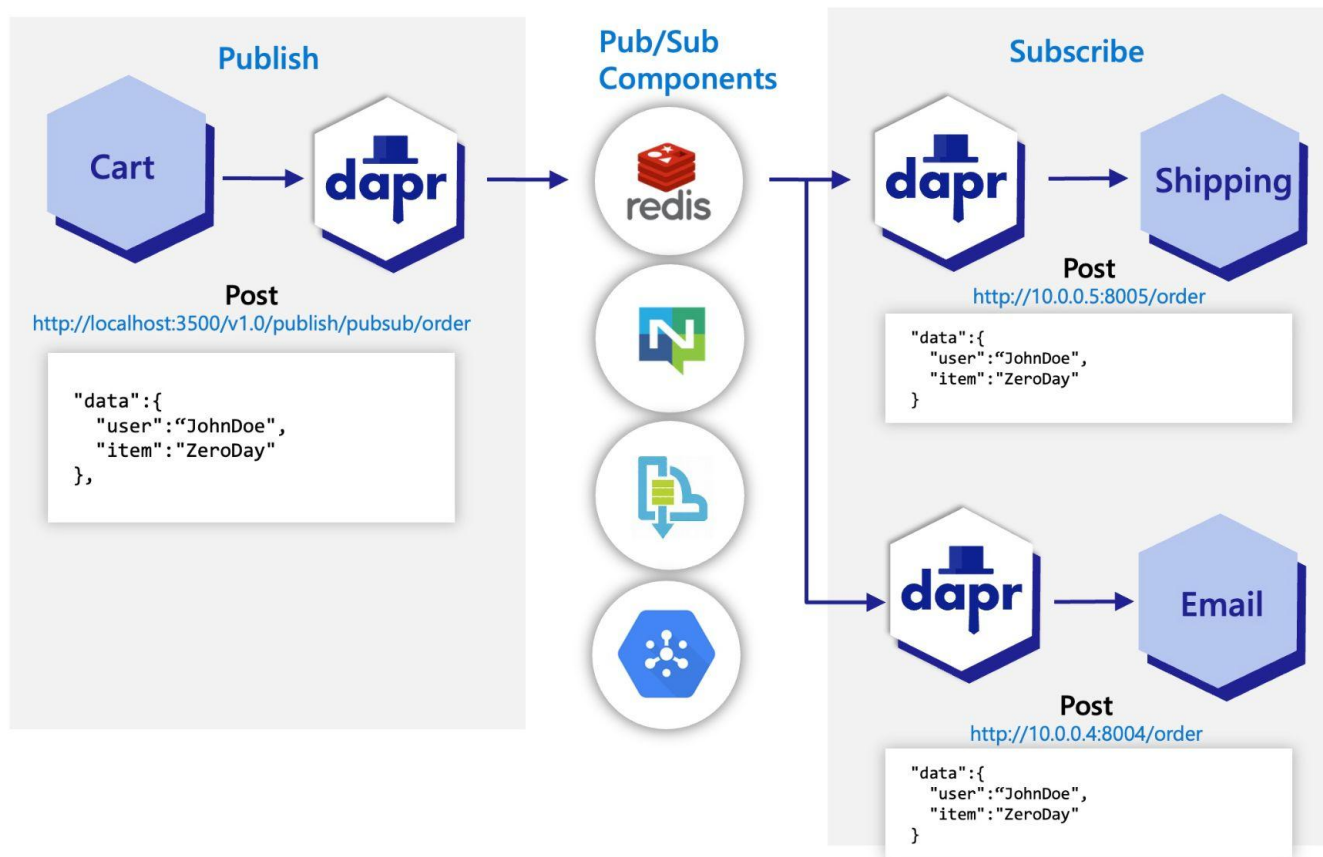
CloudNativeCon

North America 2025

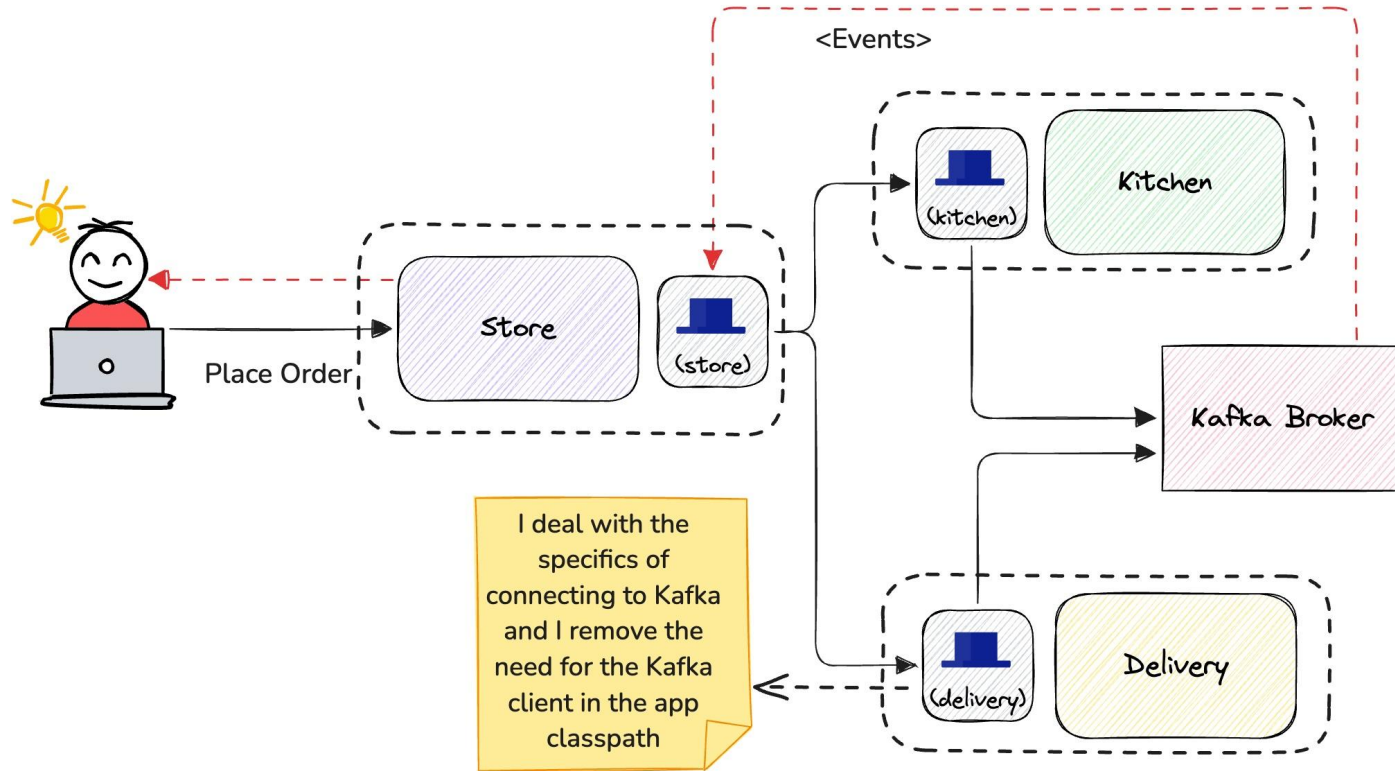
Cloud Native Pizza Store (with Dapr)



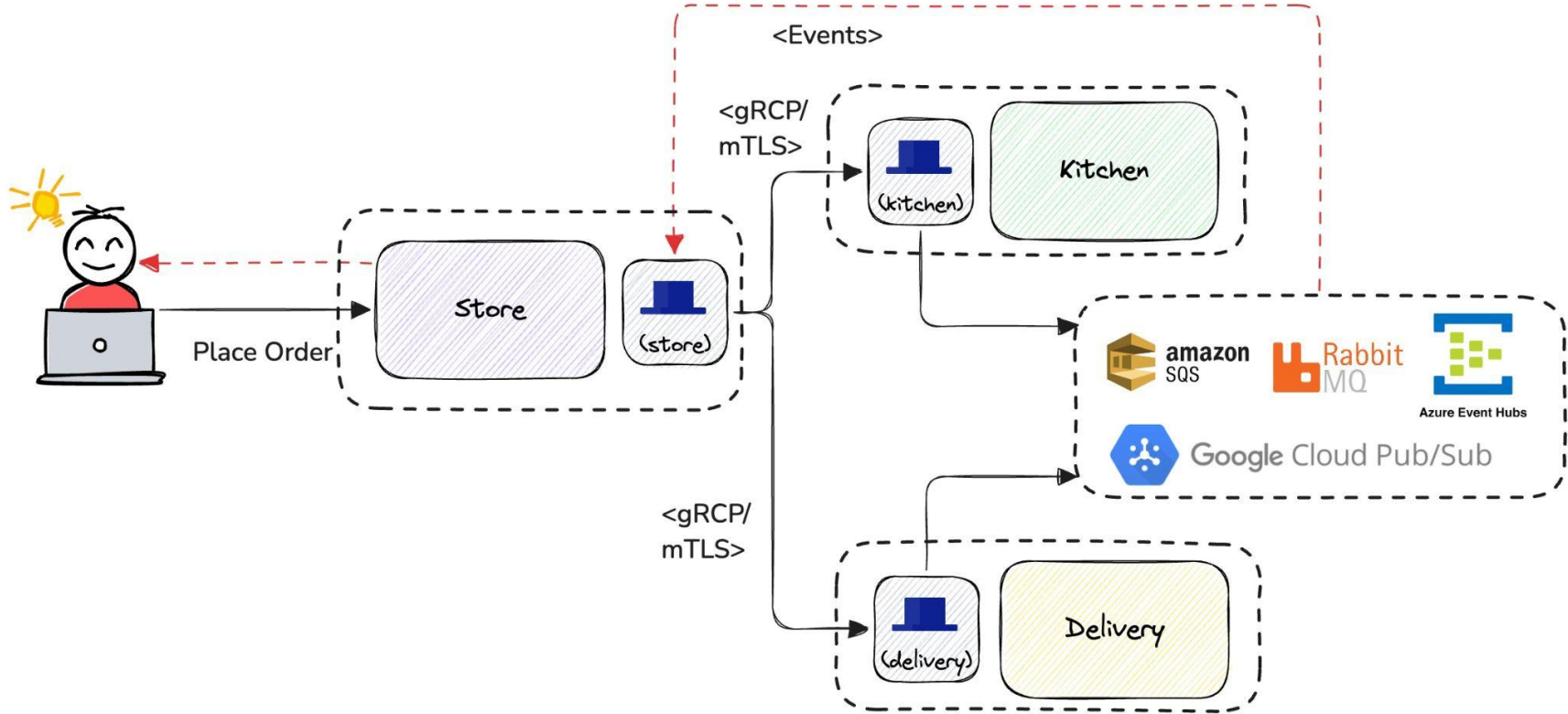
PubSub from Kitchen and Delivery to Store Service Events



Cloud Native Pizza Store (with Dapr)



Cloud Native Pizza Store (with Dapr PubSub)





KubeCon



CloudNative

North America 2025

Part #1 - Distributed app challenges

- #1 Managing cross-cutting concerns
(security, resilience, observability, ...)
- #2 Managing cross-environment concerns
(internal vs managed services, AWS vs GCP vs Azure)
- Dapr to the rescue!



Oops... I added a
new service dependency...

DevEx & DISTRIBUTED APPS - WHAT A BURDEN!



```
$ mvn spring-boot:run
```



```
$ npm run dev
```



```
$ go run main.go
```



```
$ ./gradlew bootRun
```



```
$ make run
```



```
$ ./kafka-server-start.sh
```



```
$ ./mvnw quarkus:dev
```



```
$ docker compose up -d
```



```
$ postgres -D ~/pg-data
```



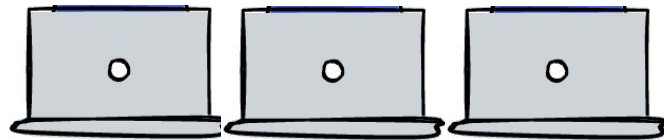
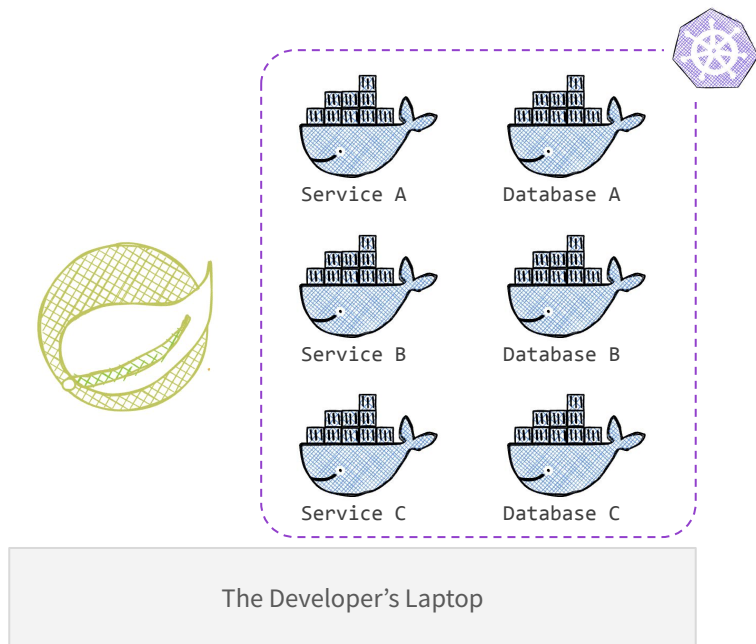
KubeCon



CloudNativeCon

North America 2025

DevEx & DISTRIBUTED APPS - WHAT THE HELL!

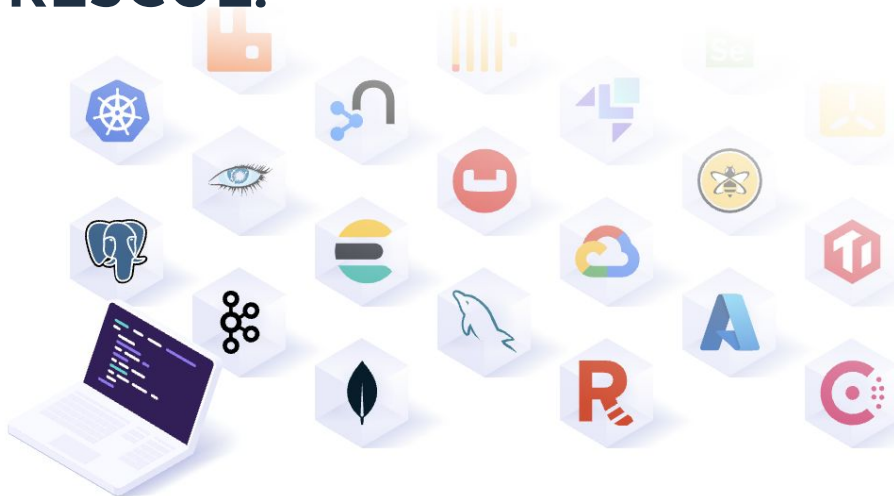


OK BUT ...
LOTS OF **COMPLEXITY**
LOTS OF **POWER / ENERGY**
LOTS OF **MONEY!**

TESTCONTAINERS to the RESCUE!

Unit tests with real dependencies

Testcontainers is an open source framework for providing throwaway, lightweight instances of databases, message brokers, web browsers, or just about anything that can run in a Docker container.



HOW IT WORKS

Test dependencies as code

No more need for mocks or complicated environment configurations. Define your test dependencies as code, then simply run your tests and containers will be created and then deleted.

With support for many languages and testing frameworks, all you need is Docker.



```
const redis = await new GenericContainer("redis:5.0.3-alpine")
  .withExposedPorts(6379)
  .withWaitStrategy(Wait.forLogMessage("Ready to accept connections"))
  .start();
```

FROM DISTRIBUTED APPS to PROMISES to ...



[Home](#)[Docs](#)[Blog](#)[Community](#)[Star](#) 1,715[Try Microcks](#)

The open source, cloud native tool for API Mocking and Testing

Microcks leverages **API standards** to provide a **uniform and multi-protocol approach**, empowering your API and microservices lifecycle. It strengthens your **ecosystem** to create an adaptable API **value chain** #APIDevOps

[Get Started](#)

🌱 1.12.1 has just been released! 🌱



[HTTPS://MICROCKS.IO](https://microcks.io)

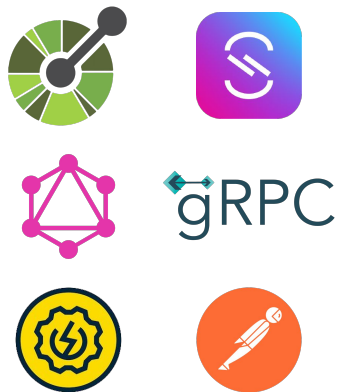


KubeCon

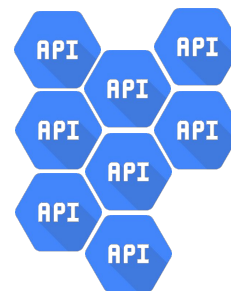
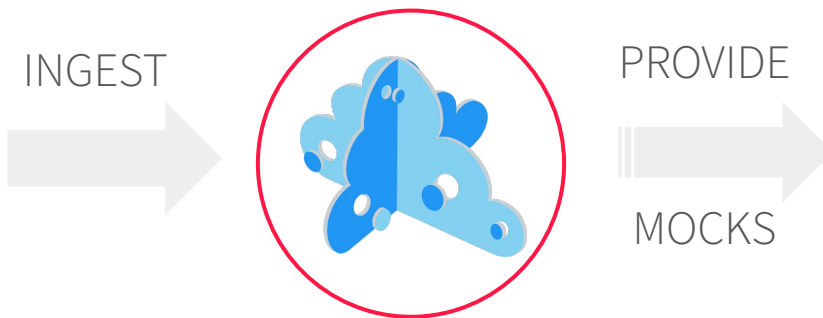
North America 2025

MICROCKS

Turn Artifacts into light & smart simulations



and others



API Consumer

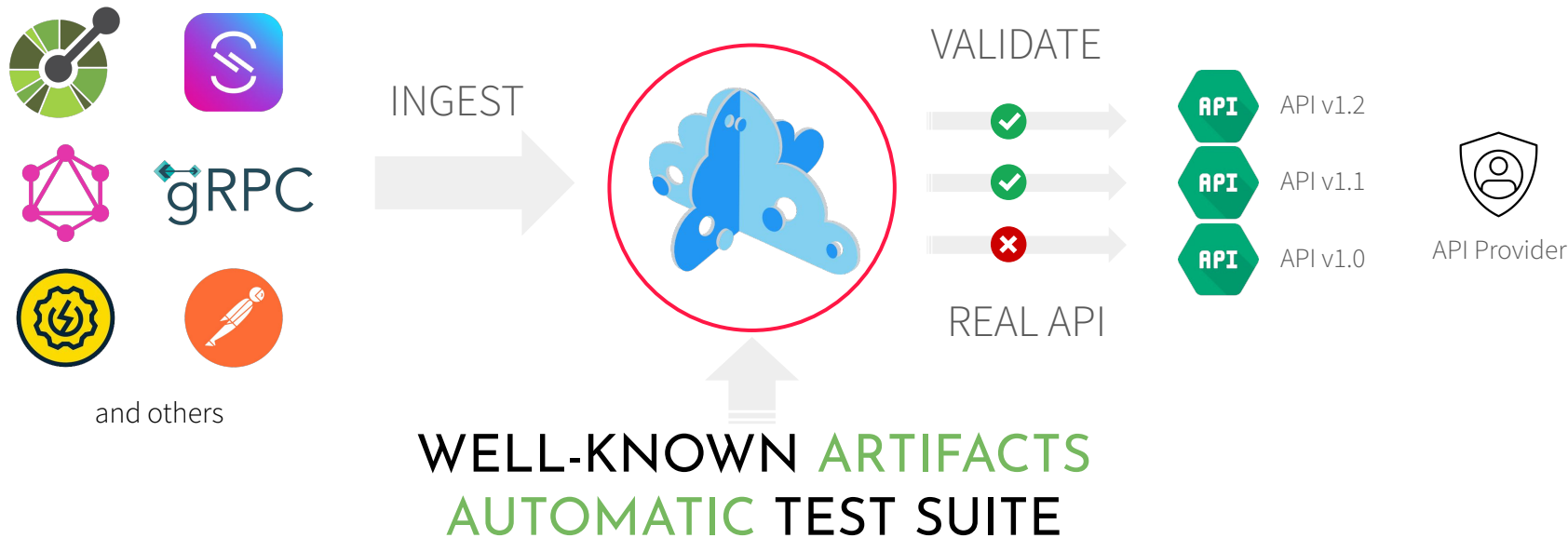
WELL-KNOWN ARTIFACTS



SMART MOCKS

MICROCKS

Turn Artifacts into Contract Testing Suite



Modules

Testcontainers modules are preconfigured implementations of various dependencies that make writing your tests even easier!

[Clear Filter](#)☐  Official Modules

Languages

All

.NET

Go

Java

Node.js

Java, NodeJS, Go, ...

**Dapr**

Cloud

Java

**Microcks**

Cloud

Java

Go

Node.js

**fake-gcs-server**

Other

Java

**MinIO**

Other

Java

Go

.NET

**SurrealDB**

NoSQL Database

Go

Rust

**Kafka**

Message Broker

Java

Go

.NET

Node.js

**OpenFGA**

Other

Java

Go

**Vault**

Other

Java

Go

Docker/AtomicJar partners with software vendors to maintain and certify official modules.



KubeCon



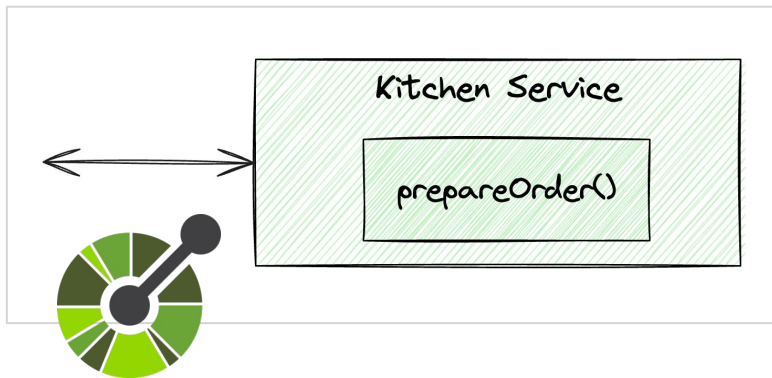
CloudNativeCon

North America 2025

Demo #2



Contract Testing in Action!

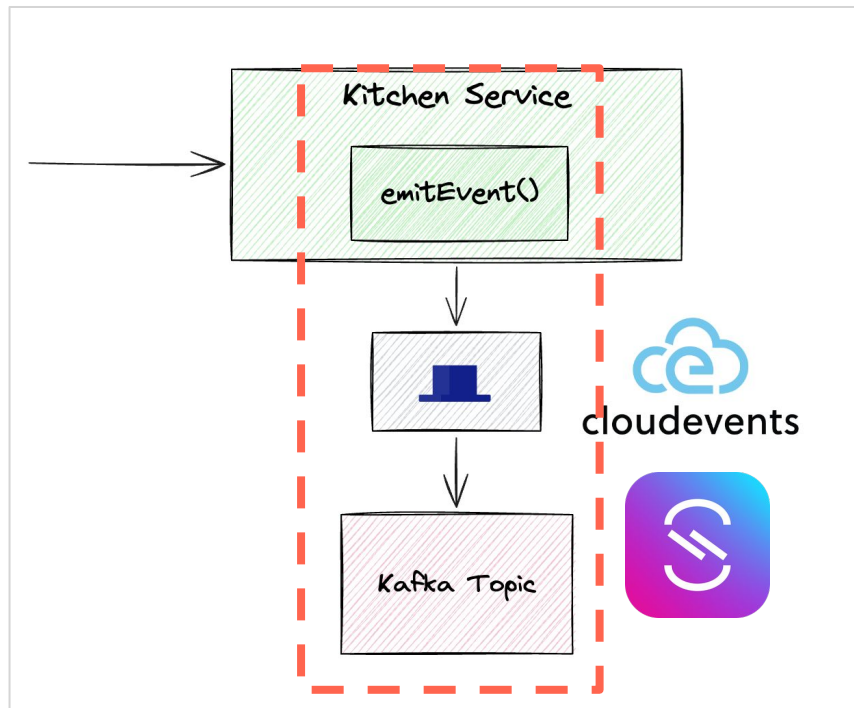


Microcks approach

- Include Microcks and OpenAPI
- Launch a Microcks Test
- For all operations and examples:
 - Act as an API consumer
 - Validate OpenAPI schema
- AssertSuccess



Contract Testing in Action for Async! 🎉



Microcks approach

- Include Microcks and AsyncAPI
- Start a Kafka broker ✨
- Launch a Microcks Test
- Invoke `KitchenService.emitEvent()`
- Check that `messagingTemplate.send()` has been called ✨
- + that message is sent over the wire
- + that message is on correct Kafka topic
- + that message is conformant to schema
- Tear-down all resources

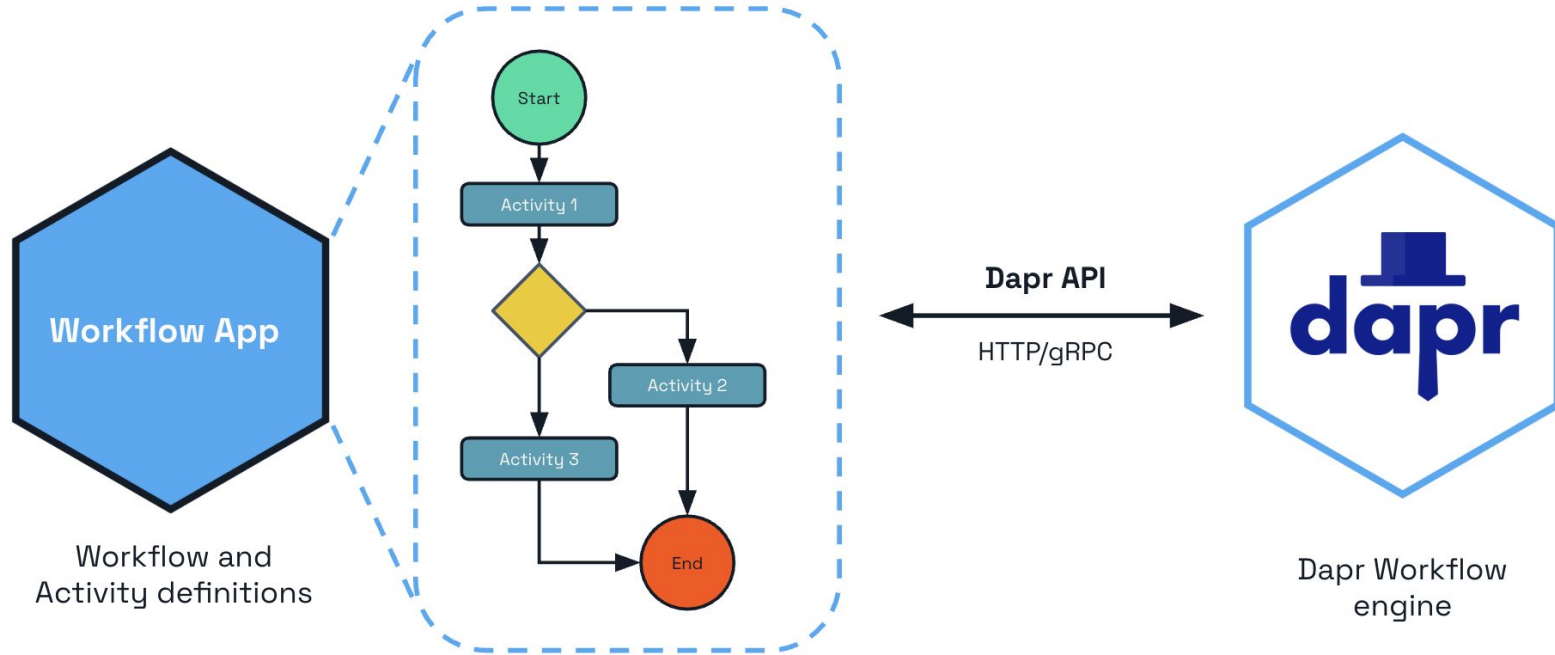


Part #2 - Distributed / DX challenges

- #3 Managing more dependencies
(complex setup, cost, slow, ...)
- #4 Testability? Confidence?
(integration tests, no breaking changes)
- Microcks to the rescue!



Dapr Workflows for Durable Executions



Workflow as Code (Workflow)

```
@Component
class ProcessPizzaOrder implements Workflow{
    run() {
        orderSaved = callActivity(SaveOrder.class, order, Order.class);
        List<Item> deliveryItems = callActivity(PlaceOrderToKitchen.class, order.items(),
                                                Item[].class);

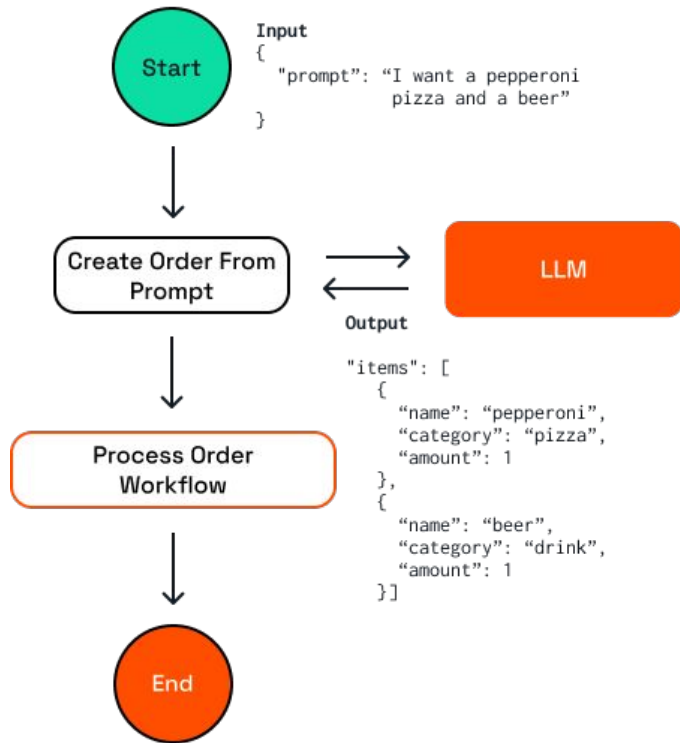
        boolean delivered = callActivity(DeliverOrderToCustomer.class, deliveryItems,
                                          Boolean.class);
        order.setIsCompleted(delivered);

        orderUpdated = callActivity(UpdateOrder.class, order, Order.class);
        completeWorkflow(orderUpdated);
    }
}
```



Order from Prompt

- Reuse workflows and activities
- Integrates with your favourite tools such as Spring AI



Demo #3



Running Ollama locally

```
2025-09-25T15:12:34.525+02:00 INFO 79735 --- [           main] o.s.a.o.management.OllamaModelManager : Start pulling model: gpt-oss:20b
2025-09-25T15:12:35.138+02:00 INFO 79735 --- [ient-3-Worker-0] o.s.a.o.management.OllamaModelManager : Pulling the 'gpt-oss:20b'
model - Status: pulling manifest
2025-09-25T15:14:55.593+02:00 INFO 79735 --- [ient-3-Worker-0] o.s.a.o.management.OllamaModelManager : Pulling the 'gpt-oss:20b'
model - Status: pulling b112e727c6f1
2025-09-25T15:14:56.883+02:00 INFO 79735 --- [ient-3-Worker-0] o.s.a.o.management.OllamaModelManager : Pulling the 'gpt-oss:20b'
model - Status: pulling fa6710a93d78
2025-09-25T15:14:58.205+02:00 INFO 79735 --- [ient-3-Worker-0] o.s.a.o.management.OllamaModelManager : Pulling the 'gpt-oss:20b'
model - Status: pulling f60356777647
2025-09-25T15:14:59.542+02:00 INFO 79735 --- [ient-3-Worker-0] o.s.a.o.management.OllamaModelManager : Pulling the 'gpt-oss:20b'
model - Status: pulling d8ba2f9a17b3
2025-09-25T15:15:00.589+02:00 INFO 79735 --- [ient-3-Worker-0] o.s.a.o.management.OllamaModelManager : Pulling the 'gpt-oss:20b'
model - Status: pulling 55c108d8e936
2025-09-25T15:15:13.851+02:00 INFO 79735 --- [ient-3-Worker-0] o.s.a.o.management.OllamaModelManager : Pulling the 'gpt-oss:20b'
model - Status: verifying sha256 digest
2025-09-25T15:15:13.861+02:00 INFO 79735 --- [ient-3-Worker-0] o.s.a.o.management.OllamaModelManager : Pulling the 'gpt-oss:20b'
model - Status: writing manifest
2025-09-25T15:15:13.866+02:00 INFO 79735 --- [ient-3-Worker-0] o.s.a.o.management.OllamaModelManager : Pulling the 'gpt-oss:20b'
model - Status: success
2025-09-25T15:15:13.867+02:00 INFO 79735 --- [           main] o.s.a.o.management.OllamaModelManager : Completed pulling the 'gpt-oss:20b' model
```

2m 40s on 1Gb internet connection!



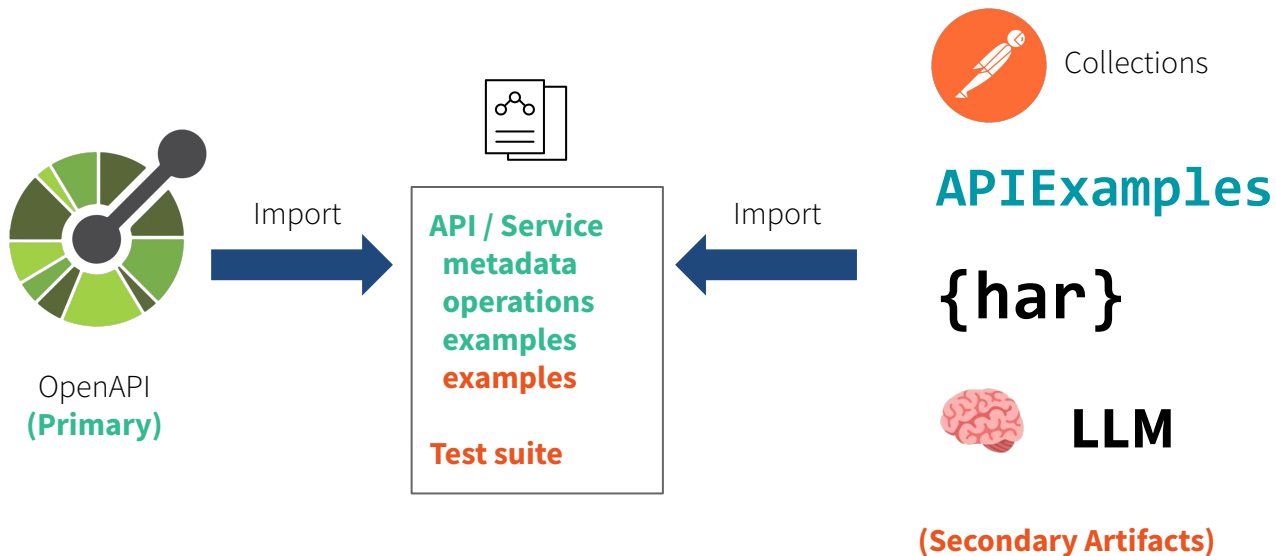
KubeCon



CloudNativeCon

North America 2025

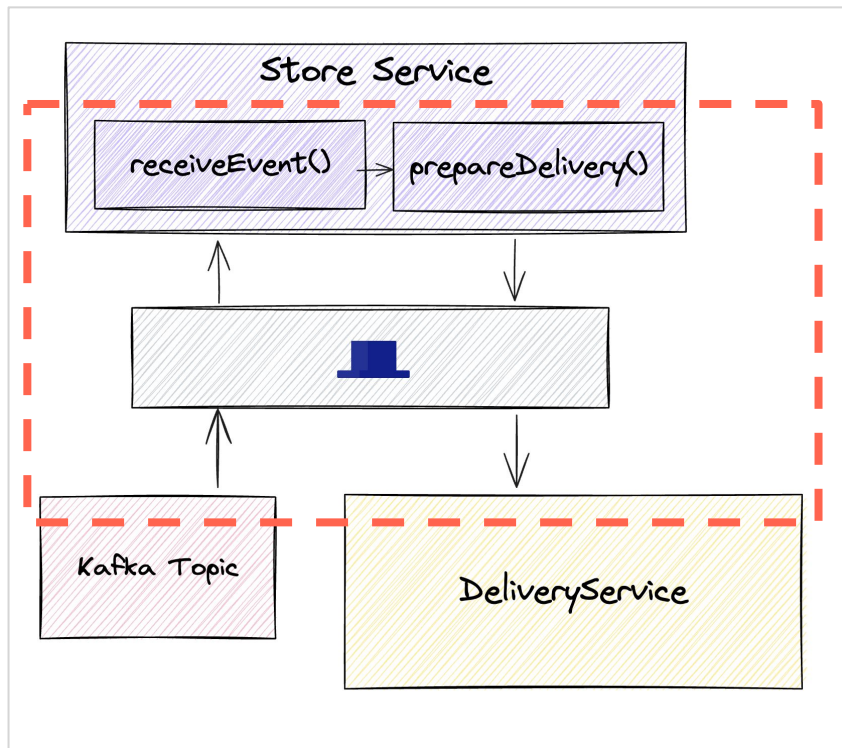
Building OpenAI mocks with Microcks!




```
# yaml-language-server: $schema= https://microcks.io/schemas/APIExamples-v1alpha1-schema.json
apiVersion: mocks.microcks.io/v1alpha1
kind: APIExamples
metadata:
  name: OpenAI API
  version: '2.3.0'
operations:
  'POST /v1/chat/completions':
    Pepperoni and Beer Order:
      request:
        headers:
          Accept: application/json
        body:
          model: gpt-4.1-2025-04-14
          messages:
            - role: user
              content: 'I would like to order a pepperoni pizza and a beer'
      response:
        mediaType: application/json
        body:
          id: chatcmpl-AyPNinnUqUDYo9SAdA52NobMflmj2
          object: chat.completion
          created: 1741569952
          model: gpt-4.1-2025-04-14
          choices:
            - index: 0
              message:
                role: assistant
                content: |-
                  [
                    {"category": "pizza", "name": "pepperoni", "amount": 1},
                    {"category": "drink", "name": "beer", "amount": 1}
                  ]
```

A simple OpenAI completion mock definition

Contract Testing in Action for Workflows! 🤖



Microcks approach

- Include Microcks and AsyncAPI
- Start a Kafka broker ✨
- Publish mock messages on Kafka
- Check that `PizzaStore.receiveEvents()` is connected and can read message
- Check that consumer deserialization is ok
- Start a workflow instance & check that `DeliveryService` is called at some point
- Tear-down all resources ✨



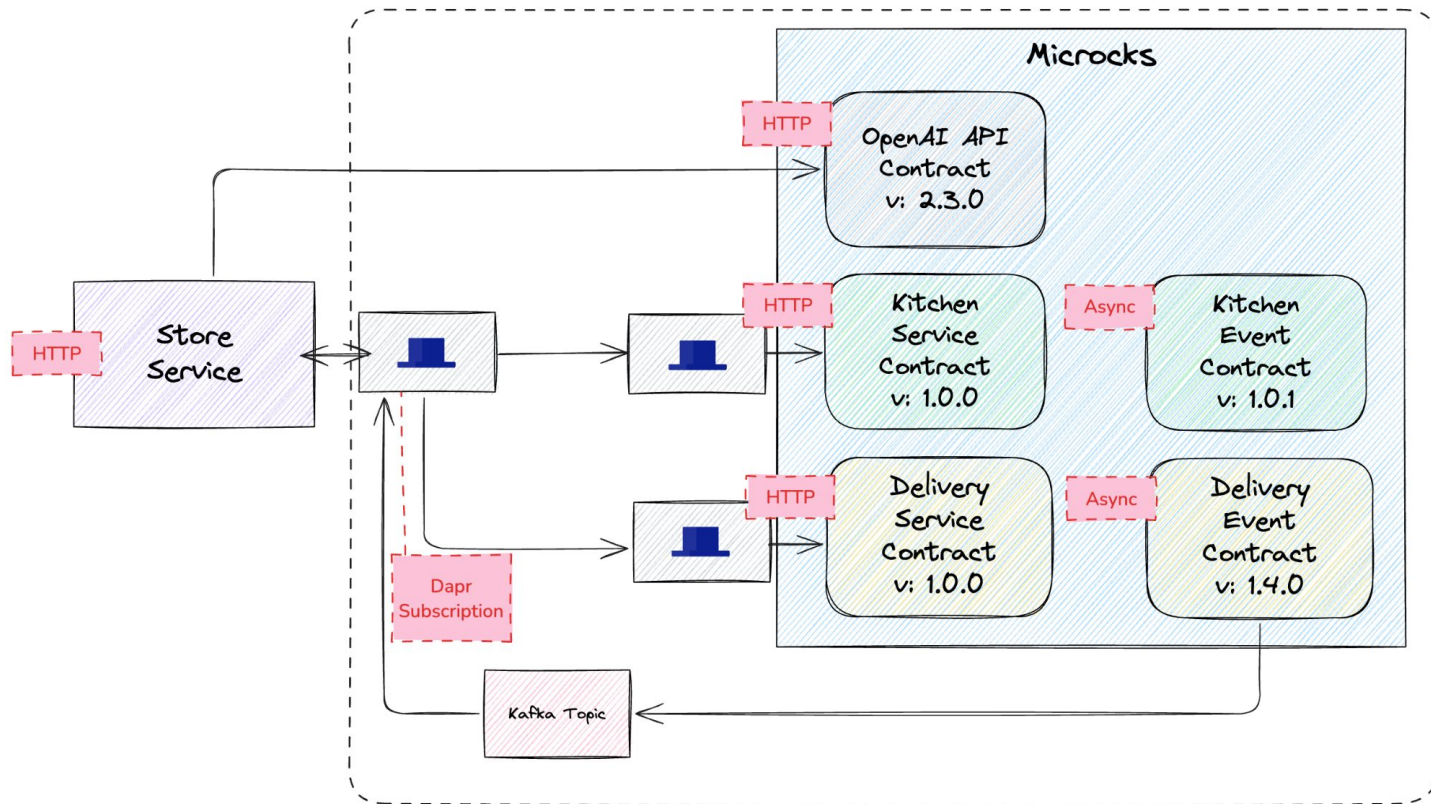


Wrap-up

ALL TOGETHER



Managed by Testcontainers



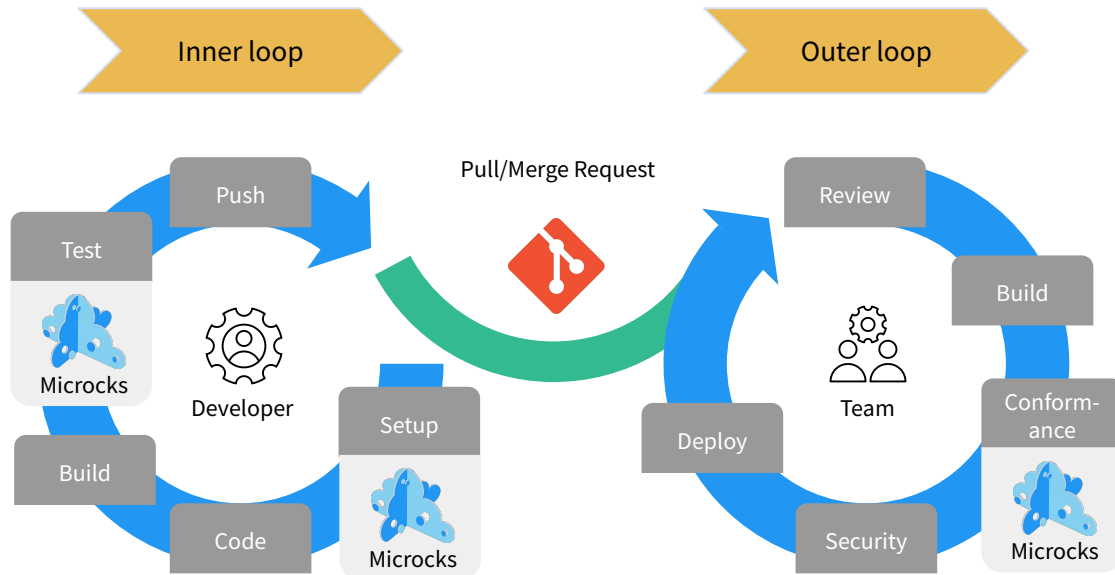
KubeCon



CloudNativeCon

North America 2025

Inner or Outer Loops: Choice is yours!



KubeCon



CloudNativeCon

North America 2025



KEY TAKEAWAYS 🤔

- Building distributed applications is hard...
- Making these apps portable across environment is possible using APIs, check out the Dapr SDK!
- Still, the Developer Experience can be a pain...
- Microcks makes mocks easy & contracts finally useful!



for a better Developer Experience,
for Java, .Net, etc.



KubeCon



CloudNativeCon

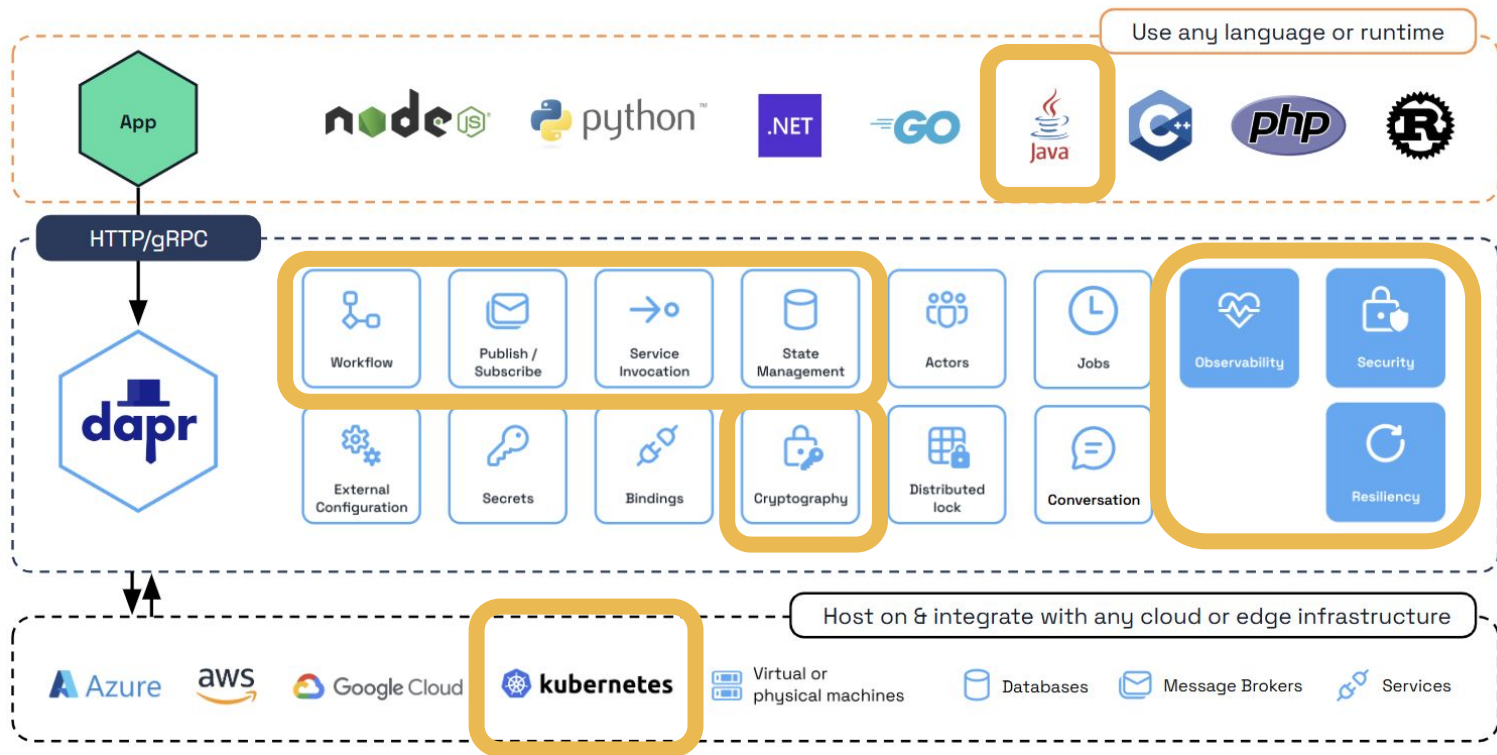
— North America 2025 —



Placing an order pseudo code

```
void placeOrder(Order order) {  
    saveOrder(order);  
  
    List<Item> deliveryItems = sendOrderToKitchen(order.items());  
  
    boolean delivered = sendItemsToCustomer(order.address(), deliveryItems);  
  
    order.setIsCompleted(delivered);  
  
    updateOrder(order)  
}
```

What we used for this demo



DevEx & DISTRIBUTED APPS - TAKING SHORTCUTS

- Avoid Testing Integrations
- Use mock frameworks **only**
 - In-VM vs Network
- Rely on **your own vision**



KubeCon

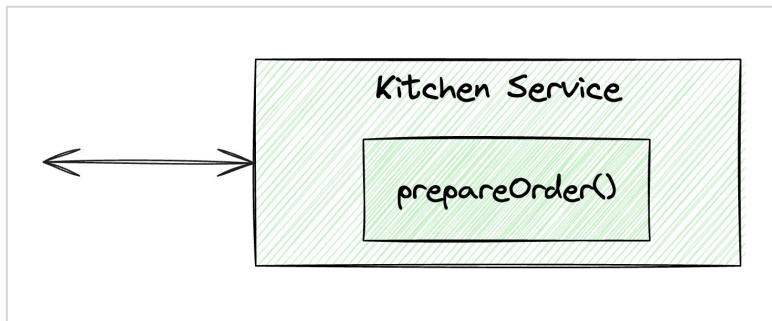


CloudNativeCon

North America 2025



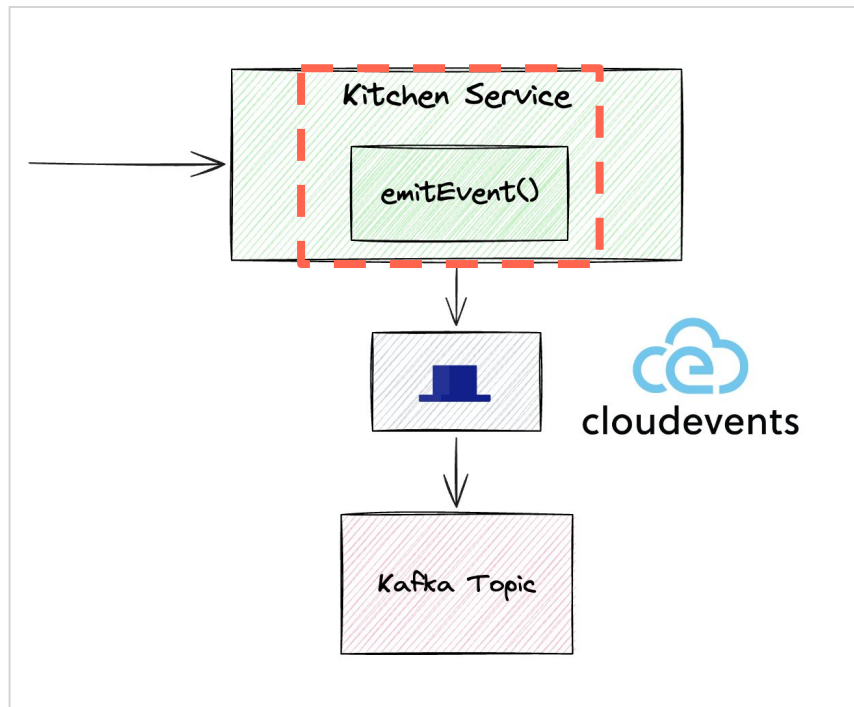
Contract Testing?



Typical approach

- Start your Application
- Include a library like RestAssured
- Call API endpoint and check:
 - Response status code
 - Response content-type
 - JSON evaluation
 - JSON response is valid on all paths
 - Loop on next call...

Contract Testing in Action for Async? 🤖



Typical "light" approach

- Include a mocking library
- Prepare Mock object for `DaprMessagingTemplate.send()`
- Check that `messagingTemplate.send()` has been called
- Tear-down all resources



KubeCon



CloudNativeCon

North America 2025

Building resiliency into our applications

```
Order placeOrder(Order order){  
    orderSaved = saveOrder(order);  
  
    List<Item> deliveryItems = sendOrderToKitchen(order.items());  
  
    boolean delivered = sendItemsToCustomer(order.address(), deliveryItems);  
  
    order.setIsCompleted(delivered);  
  
    updateOrder(order);  
  
    return orderSaved;  
}
```



KubeCon



CloudNativeCon

North America 2025

Building resiliency into our applications

```
Order placeOrder(Order order){  
  
    orderSaved = <SYNC> saveOrder(order);  
  
    <ASYNC> => List<Item> deliveryItems = sendOrderToKitchen(order.items());  
  
    <ASYNC> => boolean delivered = sendItemsToCustomer(order.address(), deliveryItems);  
  
    <ASYNC> => order.setIsCompleted(delivered);  
  
    <ASYNC> => updateOrder(order);  
  
    return orderSaved;  
  
}
```



Workflow as Code (Activity)



```
@Component
class PlaceOrderToKitchen implements WorkflowActivity {
    run(WorkflowActivityContext ctx) {
        // Call external system, for example HTTP POST to Kitchen Service
    }
}
```



KubeCon



CloudNativeCon

North America 2025

Building resiliency into our applications

```
Order placeOrder(Order order){  
    orderSaved = <SYNC>+<DURABLE> saveOrder(order);  
  
    <ASYNC>+<DURABLE> => List<Item> deliveryItems = sendOrderToKitchen(order.items());  
  
    <ASYNC>+<DURABLE> => boolean delivered = sendItemsToCustomer(order.address(), deliveryItems);  
  
    <ASYNC>+<DURABLE> => order.setIsCompleted(delivered);  
  
    <ASYNC>+<DURABLE> => updateOrder(order);  
  
    return orderSaved;  
}
```



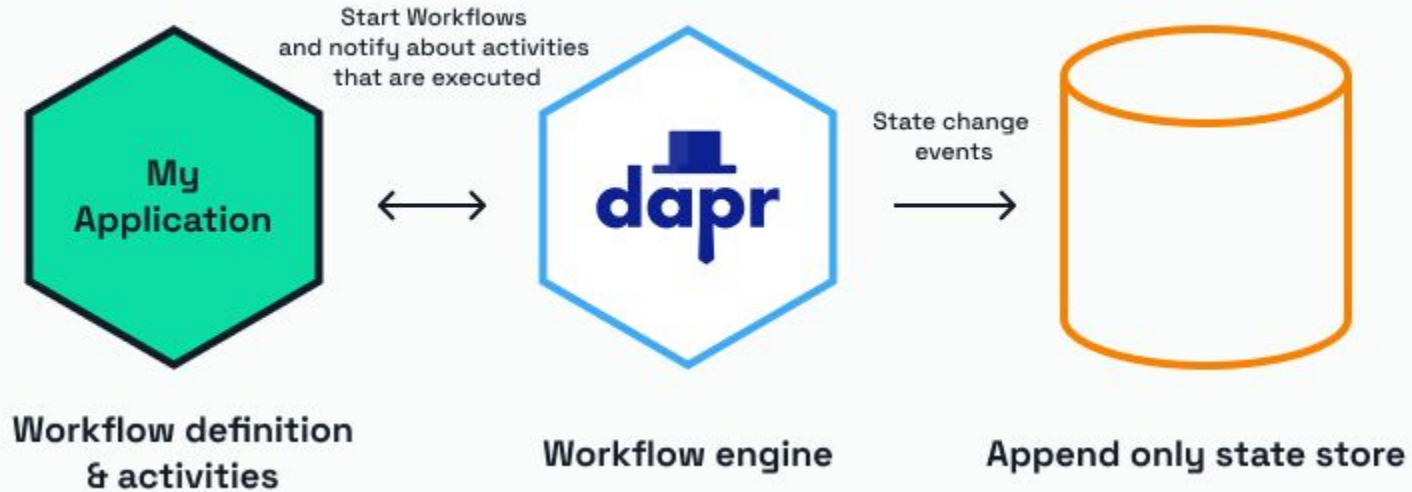
KubeCon



CloudNativeCon

North America 2025

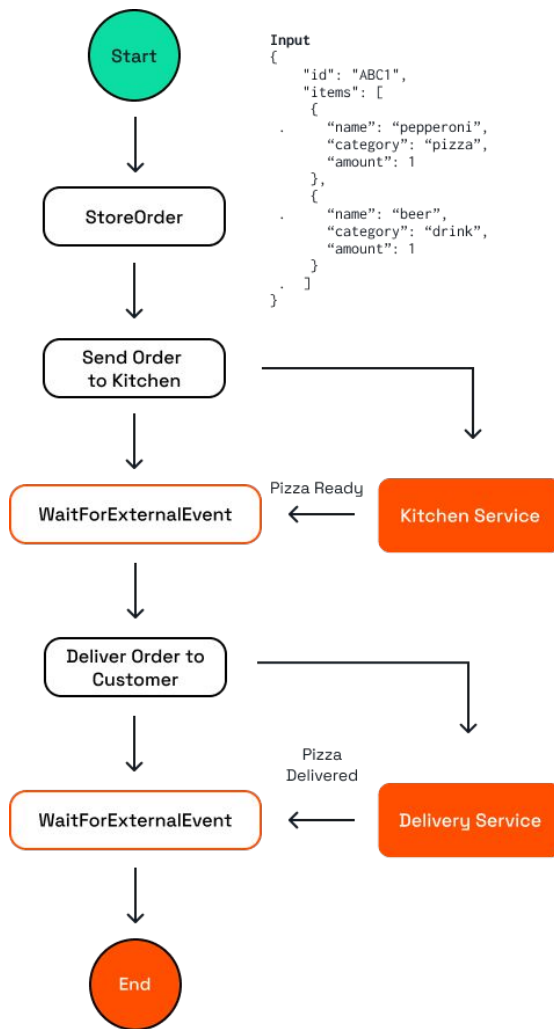
Workflow Runtime



[KubeCon 2025: Durable Executions in Serverless Architectures: Cloudflare, SpringBoot & Dapr by Nele Uhlemann, Mauricio Salatino](#)

Process Order Workflow

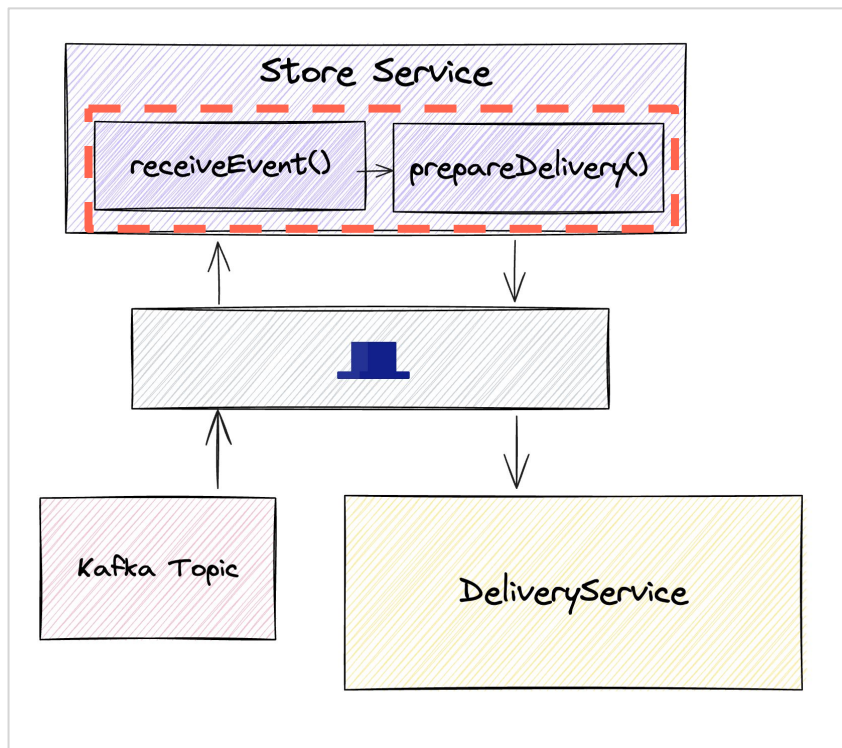
- Execute each Activity until it's done
- Retry in case of failure
- Continue from last activity executed if the application crashes



Demo #4



Contract Testing in Action for Workflows - OMG? 🤖



Typical "light" approach

- Prepare an Event object
- Invoke `PizzaStore.receiveEvents()` handling function
- Include a mocking library
- Prepare Mock object for `RestTemplate.put()`
- Check that `restTemplate.put()` has been called
- Tear-down all resources



KubeCon



CloudNativeCon

North America 2025